

Database, Modes, Announcements, and CAN Logging Test Plan

Project 6: Ryan Pratt, Nigel Sinclair, Fergus Page

Purpose

The purpose of this test plan is to provide a systematic bring-up and integration process for the Phase 2 database, modes of operation, announcements/audio, and CAN logging features.

A software simulator will be used before hardware deployment to verify system logic and identify software defects early. This allows hardware testing to focus primarily on subsystem integration, device drivers, external dependencies, and communication with physical components.

Test Scope

In Scope

The scope of this test plan covers 4 main features, the database, operating modes, announcements, and CAN logging. This includes:

- Verification of CRUD functionality in the database
- Moving and processing remote data via the database
- Integration of database thread with application
- Integration of database with modes of operation
- Announcement service and audio playback
- Operating modes function as designed
- Logging of specific CAN transmissions

Out of Scope

This test plan does not cover:

- Full stack testing
- End-to-end integration testing
- Stress testing with large volumes of traffic
- Fault handling between threads and applications

Setup

All software was tested and verified using a software simulator before deployment to the hardware. This enabled us to catch any logical errors in the code prior to deployment, saving valuable debugging time and reducing the scope of debugging from full stack to specific integration concerns. While not foolproof, the results of tests after simulation verification were significantly better.

Upon verification via simulator testing, we moved to hardware testing. In hardware testing, the software being tested would be pulled onto the raspberry pi 3 (Rpi3) built and then run through a series of tests to validate the functionality of the system as expected according to the design.

The following equipment and software are required:

- Phase 2 software simulator
- Approved Phase 2 software build
- Phase 2 hardware assembly
- Configured database
- Audio output device or speaker
- CAN interface and CAN bus simulator
- Computer with diagnostic and logging tools
- Required device drivers, libraries, and software dependencies
- Required power supply and communication cables

Before testing:

1. Install the approved software build in the simulator.
2. Load the required database, audio, and CAN configurations.
3. Confirm that simulated inputs and outputs are operating correctly.
4. Execute simulator testing before deploying the software to hardware.
5. Install and verify the required hardware drivers and dependencies.
6. Connect the CAN and audio hardware.
7. Confirm that diagnostic logs can be accessed.
8. Record the software version, hardware revision, driver versions, and dependency versions being tested.

Test Procedure

4.1 Simulator Testing

1. Start the system using the software simulator.
2. Confirm that the database, operating modes, announcements, and CAN logging functions initialize correctly.
3. Test all operating modes and permitted mode transitions.
4. Attempt invalid transitions and confirm that they are rejected.
5. Trigger simulated CAN inputs and confirm that the correct system responses occur.
6. Trigger each announcement and verify the associated system logic.
7. Verify database reads, writes, updates, and retained values.
8. Execute representative end-to-end operating scenarios.
9. Record and correct any software logic errors.
10. Repeat simulator testing until no major logical defects remain.

4.2 Hardware Bring-Up

1. Deploy the simulator-tested software build to the hardware.
2. Power on the system.
3. Confirm that all required drivers and dependencies load successfully.
4. Verify communication with the database, CAN interface, audio hardware, and other connected devices.
5. Record any startup warnings, driver failures, missing libraries, or dependency conflicts.
6. Correct configuration and integration issues before continuing functional testing.

4.3 Database Testing

1. Confirm that the database opens successfully.
2. Write a known test record.
3. Read the record and verify that the stored information is correct.
4. Update the record and confirm the change.
5. Restart the system and confirm that the data is retained.
6. Verify that database behavior on hardware matches the simulator results.

4.4 Modes-of-Operation Testing

1. Confirm that the system enters the correct default mode.
2. Activate each available operating mode.
3. Test all permitted mode transitions.
4. Confirm that invalid transitions are rejected.
5. Verify that mode behavior matches the results obtained in the simulator.
6. Confirm that mode changes correctly control the physical hardware.

4.5 Announcement Testing

1. Trigger each announcement individually.
2. Confirm that the correct audio file plays.
3. Verify that playback is clear and complete.
4. Confirm that announcements occur only in the appropriate operating modes.
5. Test announcement priority and interruption behavior.
6. Compare hardware behavior with the simulator results.

4.6 CAN Logging Testing

1. Start CAN logging.
2. Transmit known CAN messages.
3. Confirm that the correct message identifiers, data, and timestamps are recorded.
4. Compare the generated CAN log with the transmitted messages.
5. Verify that CAN-triggered system responses match the simulator results.
6. Disconnect and reconnect the CAN interface to verify fault detection and recovery.

Issues Encountered

The software simulator identified most major software logic errors before deployment to hardware. This significantly reduced the amount of logic debugging required during hardware bring-up.

The primary issues encountered during hardware testing were related to integration, including:

- Missing or incompatible device drivers
- Dependency and library version conflicts

- Differences between the simulated and physical interfaces
- Hardware communication and configuration issues
- Operating-system and deployment-environment differences
- Unexpected elevator controller faults.

Issue	Description	Resolution	Status
Audio Driver Not installed	Audio on the rpi uses pulse audio	Define header to exclude all audio backends except pulse audio	Closed
Database request not resetting	Elevator does not reset after getting database request	Clear last database request	Closed

Conclusion

The use of the software simulator was a major contributor to the largely successful completion of Phase 2. It allowed most software logic errors to be identified and corrected before deployment, reducing the risk and complexity of hardware debugging.

As a result, hardware testing was primarily focused on integration between the software and physical system. The most significant hardware bring-up challenges involved device drivers, software dependencies, and interface configuration rather than core application logic.

Phase 2 testing will be considered successful when:

- Major software logic has been verified using the simulator.
- Simulator results are successfully reproduced on hardware.
- The database operates correctly and retains valid data.
- All operating modes and transitions function as designed.
- Announcements are triggered and played correctly.
- CAN messages are logged accurately.
- Required drivers and dependencies operate reliably.
- No unresolved critical integration issues remain.